



Topic:

Figma Framework — Mockup → Test Cases (Shift-Left QA)

Scope:

The Figma Framework produces manual test cases *directly from design mockups* — before a single line of code has been written. Given a Figma file key or full URL, it fetches the file JSON via the Figma REST API v1, walks the document tree to collect every top-level frame (Login, Signup, Forgot Password, etc.), requests PNG renders at 2× scale from the image-export endpoint, and downloads them locally. `figmaToMarkdown.buildBrief()` turns the file JSON + frames into a compact Markdown brief, which is sent to Anthropic Claude (vision) together with the PNG images as multipart input. Claude returns a structured JSON of test cases across six enforced categories. `validator.js` rejects malformed output, `versionStore` archives every run, and ExcelJS writes per-feature sheets so QA can review coverage with the designer while the screen is still on paper.

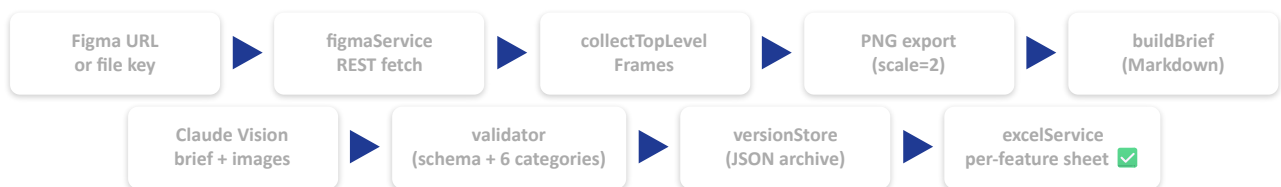
Key Features of the Figma Framework:

- **Vision LLM:** Claude (Sonnet 4.6 multimodal) reads PNG frames + Markdown brief in a single multipart prompt — text and image together; one frame caption per image.
- **Cache-First Architecture:** Cached `figma-files/<key>.json` + `frames/<key>/` reused on reruns — zero Figma API spend after first fetch. `FIGMA_LOCAL_ONLY=1` makes a cache miss a hard error (offline mode).
- **Six Enforced Test Categories:** UI/Layout, Functional, Negative, Validation, Accessibility, Responsive — coverage guaranteed on every frame; ~30–60 cases for a 3–5 screen file.
- **Two Figma Endpoints:** `/v1/files/<key>` (document JSON) + `/v1/images/<key>?ids=...&format=png&scale=2` (signed PNG URLs, downloaded via native fetch).
- **Per-Feature Excel Sheet:** Each Figma file's cases land in their own sheet (sheet name = the `feature` field Claude returns); re-runs replace just that sheet — other features' sheets untouched.
- **Versioned JSON Archive:** Every LLM output timestamped to `output/json/<key>_<ts>.json` — full audit trail before any Excel write.
- **Configurable Render Scale:** `FIGMA_EXPORT_SCALE` (default 2×) sharpens text in the PNG so the vision model OCRs less and reasons more about layout/states.
- **Hard Cap:** `FIGMA_MAX_FRAMES` protects huge files from runaway image exports + LLM cost.
- **URL OR Key Input:** `parseFileKey()` accepts a raw key or a full Figma URL; multiple files in one invocation (`node index.js KEY1 KEY2`).

Why Use the Figma Framework?

Traditional test-case writing waits for a build — QA can't write meaningful cases until they can click on something. By that point the design is frozen and any coverage gap surfaces as a late bug or a missed sprint. The Figma Framework moves QA **left of the build**: the moment a screen exists in Figma, manual test cases exist in Excel. Designers spot missing states, PMs spot missing acceptance criteria, and the test cases the Jira and Swagger frameworks need later are already drafted and reviewed. The six-category coverage rule guarantees no frame ships without UI/Layout, Functional, Negative, Validation, Accessibility and Responsive cases. Cache-first + local-only mode keep Figma API spend near zero on iteration.

Figma Framework Architecture



Integration with Master Orchestrator, Tooling & CI/CD

Invoked via `node index.js --framework=figma`, `figmaAdapter.js` reuses the orchestrator's shared cache at `master-orchestrator/inputs/figma/` with `FIGMA_LOCAL_ONLY=1` so multiple frameworks share a single source-of-truth. Built on **Node.js + Anthropic Claude (Vision) + Figma REST API v1 + native fetch + ExcelJS + dotenv**. Required `.env`: `FIGMA_ACCESS_TOKEN`, `ANTHROPIC_API_KEY`, `LLM_MODEL`; optional knobs: `FIGMA_EXPORT_SCALE`, `FIGMA_MAX_FRAMES`, `FIGMA_LOCAL_ONLY`, `FIGMA_FILES_DIR`, `FIGMA_FRAMES_DIR`. Output Excel files are CI-friendly and integrate with the Master Orchestrator's consolidated summary.

